Indian Institute of Technology Hyderabad
Department of Computer Science and Engineering

Programming Assignment 2

DNS and DNSSEC

CS6903: Network Security

Aayush Ranjan
Roll No: CS25MTECH11002

ANTI-PLAGIARISM STATEMENT

I certify that this assignment/report is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication. I also certify that this assignment/report has not previously been submitted for assessment in any other course, and that I have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. I pledge to uphold the principles of honesty and responsibility at CSE@IITH.

Name: Aayush Ranjan **Roll No:** CS25MTECH11002

Date: 22 April 2026 **Signature:** AR

Contents

1 Environment Setup **4**
1.1 System Description . . . 4
1.2 Tools and Dependencies . . . 4
1.3 Installation Steps . . . 4
1.3.1 dnspython Installation . . . 4
1.3.2 Docker Desktop Installation . . . 4
1.3.3 Rosetta Emulation . . . 5
1.3.4 SEED Lab Setup . . . 5
1.4 Project Folder Structure . . . 5

2 Q1: DNSSEC Validation Module **6**
2.1 Overview . . . 6
2.2 Approach . . . 6
2.3 Key Design Decisions . . . 6
2.4 Code . . . 7
2.4.1 Function Descriptions . . . 7
2.4.2 Chain of Trust Diagram . . . 8
2.4.3 Error Handling . . . 8
2.5 Output . . . 9
2.5.1 Test 1: example.com (A record) . . . 9
2.5.2 Test 2: cloudflare.com (A record) . . . 10
2.6 Observations . . . 10

3 Q2: Recursive Resolver with DNSSEC Validation **11**
3.1 Overview . . . 11
3.2 Approach . . . 11
3.2.1 Resolution Path . . . 11
3.2.2 Resolution Algorithm . . . 11
3.3 Key Design Decisions . . . 11
3.4 Code . . . 12
3.5 Output . . . 13
3.5.1 Test 1: example.com – Part 1 (Root and .com zones) . . . 13
3.5.2 Test 1: example.com – Part 2 (example.com zone and final result) 14
3.5.3 Test 2: cloudflare.com – Part 1 (Root and .com zones) . . . 15
3.5.4 Test 2: cloudflare.com – Part 2 (cloudflare.com zone and final result) 16
3.6 Observations . . . 17

4 Q3: NSEC/NSEC3 Non-existence Handler **18**
4.1 Overview . . . 18
4.2 Background . . . 18
4.2.1 NSEC (Next Secure Record) . . . 18
4.2.2 NSEC3 (Next Secure Record version 3) . . . 18
4.2.3 Two Cases of Non-existence . . . 18
4.3 Approach . . . 18
4.4 Key Design Decisions . . . 19
4.5 Code . . . 19

4.6 Output . . . 20
4.6.1 Test 1: mail.example.com TXT – NODATA (NSEC) . . . 20
4.6.2 Test 2: nonexistent12345.com A – NXDOMAIN (NSEC3) . . . 21
4.6.3 Test 3: example.com TXT – EXISTS . . . 22
4.7 Observations . . . 22

5 Q4: DNSSEC Key Lifecycle Analysis 23
5.1 Overview . . . 23
5.2 Background: DNSSEC Key Lifecycle . . . 23
5.3 Approach . . . 23
5.4 Key Design Decisions . . . 23
5.5 Code . . . 24
5.6 Output . . . 25
5.6.1 Test 1: example.com – ZSK Rollover in Progress . . . 25
5.6.2 Test 2: cloudflare.com – Normal . . . 26
5.6.3 Test 3: verisign.com – Normal (Algorithm 8) . . . 27
5.7 Observations . . . 28

6 Q5: DNSSEC Tampering Detection 29
6.1 Overview . . . 29
6.2 Part A: Environment Setup . . . 29
6.3 Part B: Tampering with Records . . . 30
6.4 Part C: Query and Verification . . . 31
6.4.1 Before Tampering – dig output . . . 31
6.4.2 After Tampering – dig output . . . 31
6.4.3 Local DNS Server Query – After Tampering . . . 32
6.4.4 Custom Validator – Before Tampering . . . 33
6.4.5 Custom Validator – After Tampering . . . 33
6.5 Part D: Analysis . . . 34
6.5.1 1. Response returned by the resolver . . . 34
6.5.2 2. The ad (Authenticated Data) flag . . . 34
6.5.3 3. Custom validator response . . . 34
6.5.4 4. Exact failure point . . . 34
6.5.5 5. Why DNSSEC validation fails . . . 35
6.6 Observations . . . 35

1 Environment Setup

1.1 System Description

The following system was used for this assignment:

Component	Details
Machine	MacBook Air M4 (Apple Silicon, ARM64)
Operating System	macOS (Apple Silicon)
Python Version	3.9.6
dnspython Version	2.7.0
Docker Version	29.4.0
IDE	Visual Studio Code

Table 1: System Configuration

1.2 Tools and Dependencies

The following tools were installed and configured:

- **Python 3.9.6** – Used for implementing DNSSEC validation modules (Q1–Q4)
- **dnspython 2.7.0** – Python library for DNS queries and record parsing
- **Docker Desktop 29.4.0** – Used to run the SEED Lab containerized DNS infrastructure for Q5
- **Rosetta x86/amd64 emulation** – Enabled in Docker Desktop to support SEED Lab containers on Apple Silicon
- **bind9utils** – Provides `dnssec-keygen` and `dnssec-signzone` tools (used inside containers)

1.3 Installation Steps

1.3.1 dnspython Installation

```
pip3 install dnspython
# Verify
python3 -c "import dns.resolver; print('dnspython OK')"
```

1.3.2 Docker Desktop Installation

```
brew install --cask docker --force
# Verify
docker --version
# Output: Docker version 29.4.0, build 9d7ad9f
```

1.3.3 Rosetta Emulation

Rosetta for x86_64/amd64 emulation was enabled in Docker Desktop under:
`Settings → General → Use Rosetta for x86_64/amd64 emulation on Apple Silicon`

1.3.4 SEED Lab Setup

The SEED DNSSEC Lab repository was cloned and the ARM-specific lab setup was copied:

```
git clone https://github.com/seed-labs/seed-labs.git
cp -r seed-labs/category-network/DNSSEC/Labsetup-arm/ \
      q5_tampering_demo/Labsetup-arm/
```

1.4 Project Folder Structure

```
\begin{lstlisting}
CS25MTECH11002/
    q1_validation_module/
        dnssec_validator.py
    q2_recursive_resolver/
        recursive_resolver.py
    q3_nsec_handling/
        nsec_handler.py
    q4_key_lifecycle/
        key_lifecycle.py
    q5_tampering_demo/
        q5_validator.py
        Labsetup-arm/
            base_image/
            local_dns_server/
            nameserver/
            user/
            docker-compose.yml
    report/
        images/
        report.pdf
    README.md
```

2 Q1: DNSSEC Validation Module

2.1 Overview

A reusable DNSSEC validation module was implemented in Python using the `dnspython` library. The module takes a domain name and record type as input, retrieves all necessary DNS records, and performs a full DNSSEC chain-of-trust validation.

2.2 Approach

The validation is broken into six sequential steps:

1. **Fetch Answer Records** – Retrieve the actual DNS records (A, AAAA, etc.) for the domain using a DNSSEC-aware resolver pointed at Google's public DNS (8.8.8.8).
2. **Fetch DNSKEY Records** – Retrieve the Zone Signing Key (ZSK, flags=256) and Key Signing Key (KSK, flags=257) published in the domain's zone. These are the public keys used for signature verification.
3. **Fetch RRSIG Records** – Retrieve the digital signature over the requested record type. The RRSIG is created by the zone owner using their ZSK private key.
4. **Fetch DS Record from Parent** – Retrieve the Delegation Signer (DS) record from the parent zone. The DS record contains a cryptographic hash of the child zone's KSK, anchoring the chain of trust.
5. **Verify RRSIG using DNSKEY (ZSK)** – Use `dns.dnssec.validate()` to verify the RRSIG signature against the DNSKEY. This confirms the DNS records were signed by the legitimate zone owner.
6. **Verify DNSKEY using DS (Chain Step)** – Compute the DS digest of the KSK using `dns.dnssec.make_ds()` and compare it against the DS record from the parent zone. A match confirms the KSK is trusted by the parent.

2.3 Key Design Decisions

- The module is designed to be **fully reusable** – the `validate_dnssec(domain, record_type)` function returns a structured dictionary that Q2, Q3, and Q4 can import and call directly.
- EDNS with the DNSSEC OK (DO) bit is set on all queries to ensure DNSSEC records are returned by the resolver.
- The `cryptography` package is used internally by `dnspython` for elliptic curve and RSA signature verification.

2.4 Code

The module is structured into six independent functions, each handling one step of the validation pipeline. This modular design allows Q2, Q3, and Q4 to import and reuse individual functions as needed.

Listing 1: dnssec_validator.py (key functions)

```python
def validate_dnssec(domain, record_type):
    # Step 1: Get answer records
    answer = get_answer_records(domain, record_type)
    # Step 2: Get DNSKEY (ZSK + KSK)
    keys, dnskey_answer = get_dnskey(domain)
    # Step 3: Get RRSIG
    answer_rrset, rrsig_list = get_rrsig(domain, record_type)
    # Step 4: Get DS from parent zone
    ds_answer = get_ds_from_parent(domain)
    # Step 5: Verify RRSIG using ZSK
    rrsig_valid = verify_rrsig(domain, answer_rrset, rrsig_list,
        keys)
    # Step 6: Verify KSK using DS
    ds_valid = verify_dnskey_with_ds(domain, dnskey_answer,
        ds_answer)
    return result
```

2.4.1 Function Descriptions

- `make_resolver()` – Creates a DNS resolver instance with EDNS and the DNSSEC OK (DO) bit enabled, pointing to Google's public DNS at 8.8.8.8. This ensures all queries return DNSSEC-related records alongside normal answers.
- `get_answer_records(domain, record_type)` – Sends a standard DNS query for the requested record type. Returns the full answer RRset which is later used in RRSIG verification.
- `get_dnskey(domain)` – Fetches all DNSKEY records for the zone. Identifies each key as either a ZSK (flags=256) or KSK (flags=257) and returns both the raw answer and a structured keys dictionary for use in `dns.dnssec.validate()`.
- `get_rrsig(domain, record_type)` – Sends a raw DNS query with the DO bit set and extracts RRSIG records from the answer section. The RRSIG contains the digital signature, key tag, algorithm, and signer name.
- `get_ds_from_parent(domain)` – Queries the parent zone for the DS record of the given domain. The DS record holds a cryptographic digest of the child zone's KSK, forming the delegation anchor in the chain of trust.
- `verify_rrsig(domain, answer_rrset, rrsig_list, keys)` – Uses `dns.dnssec.validate()` to cryptographically verify the RRSIG signature against the DNSKEY. Internally uses the `cryptography` library for ECDSA/RSA operations.

- `verify_dnskey_with_ds(domain, dnskey_answer, ds_answer)` – Iterates over all KSK records and computes their DS digest using `dns.dnssec.make_ds()`. Compares the computed digest against the DS record retrieved from the parent zone. A match confirms the chain of trust.

2.4.2 Chain of Trust Diagram

The DNSSEC chain of trust validated by this module follows this structure. Each level of the hierarchy vouches for the level below it, creating an unbroken chain from the globally trusted root all the way down to the individual DNS record being queried.

```
Root Zone (.)
    |
    |-- DS record for .com  (root signs .com KSK)
    v
.com TLD Zone
    |
    |-- DS record for example.com  (.com signs example.com KSK)
    v
example.com Zone
    |
    |-- KSK signs ZSK  (verified by DS from parent)
    |-- ZSK signs A record  (verified by RRSIG)
    v
Final Answer: A record (IP address) -- TRUSTED
```

2.4.3 Error Handling

The module includes robust error handling at every step:

- **NXDOMAIN** – If the domain does not exist, the module catches `dns.resolver.NXDOMAIN` and reports it cleanly without crashing. This is important for Q3 which handles non-existent domains.
- **NoAnswer** – If a record type does not exist for a domain (e.g. querying AAAA for a domain with no IPv6), the module catches `dns.resolver.NoAnswer` and reports the specific missing record.
- **Missing RRSIG** – If no RRSIG is found, the module reports that the zone is not DNSSEC signed and halts validation cleanly.
- **Validation Failure** – If `dns.dnssec.validate()` raises `dns.dnssec.ValidationFailure`, the exact failure reason is captured and reported. This is the primary detection mechanism for tampered records (used in Q5).
- **DS Mismatch** – If no DS record in the parent matches any KSK in the child zone, the module reports a chain of trust failure. This detects KSK rollover issues (used in Q4).

2.5 Output

The module was tested against two real-world DNSSEC-signed domains: `example.com` and `cloudflare.com`. Both domains use Algorithm 13 (ECDSA P-256 with SHA-256), which is the current recommended algorithm for DNSSEC. The complete terminal output for each test is shown below, demonstrating successful validation at every step of the chain.

2.5.1 Test 1: example.com (A record)

Figure 1 shows the complete terminal output for validating `example.com` with record type A.

```
(base) rnjn_aayush@Aayushs-MacBook-Air q1_validation_module % python3 dnssec_validator.py example.com A
  ============================================================
    DNSSEC Validation
    Domain : example.com
    Record : A
  ============================================================

  [*] Fetching A records for example.com...
  [+] Found 2 A record(s):
      example.com -> 172.66.147.243
      example.com -> 104.20.23.154

  [*] Fetching DNSKEY records for example.com...
  [+] Found 4 DNSKEY record(s):
      [KSK] flags=257 algorithm=13 key_tag=2371
      [ZSK] flags=256 algorithm=13 key_tag=9776
      [ZSK] flags=256 algorithm=13 key_tag=34505
      [ZSK] flags=256 algorithm=13 key_tag=36315

  [*] Fetching RRSIG for example.com A...
  [+] Found RRSIG:
      covers=A algorithm=13 key_tag=34505 signer=example.com.

  [*] Fetching DS record for example.com from parent zone...
  [+] Found 1 DS record(s):
      key_tag=2371 algorithm=13 digest_type=2 digest=c988ec423e3880eb8dd8...

  [*] Verifying RRSIG using DNSKEY (ZSK)...
  [+] RRSIG verified successfully using ZSK.

  [*] Verifying DNSKEY using DS record (chain of trust)...
  [+] DS record matches KSK (key_tag=2371).
  [+] Chain of trust verified: Parent DS → Child KSK.

  ============================================================
    Domain: example.com
    Record: A
    DNSSEC Validation: VALID

    Steps:
      • DNSKEY retrieved
      • RRSIG verified using ZSK
      • DS matched parent
  ============================================================
```

Figure 1: DNSSEC Validation output for example.com – VALID

2.5.2 Test 2: cloudflare.com (A record)

Figure 2 shows the complete terminal output for validating `cloudflare.com` with record type A.

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q1_validation_module % python3 dnssec_validator.py cloudflare.com A
  ============================================================
    DNSSEC Validation
    Domain : cloudflare.com
    Record : A
  ============================================================

  [*] Fetching A records for cloudflare.com...
  [+] Found 2 A record(s):
      cloudflare.com -> 104.16.133.229
      cloudflare.com -> 104.16.132.229

  [*] Fetching DNSKEY records for cloudflare.com...
  [+] Found 2 DNSKEY record(s):
      [KSK] flags=257 algorithm=13 key_tag=2371
      [ZSK] flags=256 algorithm=13 key_tag=34505

  [*] Fetching RRSIG for cloudflare.com A...
  [+] Found RRSIG:
      covers=A algorithm=13 key_tag=34505 signer=cloudflare.com.

  [*] Fetching DS record for cloudflare.com from parent zone...
  [+] Found 1 DS record(s):
      key_tag=2371 algorithm=13 digest_type=2 digest=32996839a6d808afe3eb...

  [*] Verifying RRSIG using DNSKEY (ZSK)...
  [+] RRSIG verified successfully using ZSK.

  [*] Verifying DNSKEY using DS record (chain of trust)...
  [+] DS record matches KSK (key_tag=2371).
  [+] Chain of trust verified: Parent DS → Child KSK.

  ============================================================
    Domain: cloudflare.com
    Record: A
    DNSSEC Validation: VALID

    Steps:
      • DNSKEY retrieved
      • RRSIG verified using ZSK
      • DS matched parent
  ============================================================
```

Figure 2: DNSSEC Validation output for cloudflare.com – VALID

2.6 Observations

- Both `example.com` and `cloudflare.com` are fully DNSSEC signed and validated successfully.
- `example.com` had 4 DNSKEY records (1 KSK + 3 ZSKs), indicating multiple active signing keys – a common practice for key rollovers.
- Algorithm 13 (ECDSA with P-256 and SHA-256) was used by both domains, which is the modern standard for DNSSEC.
- The DS record in the parent zone matched the KSK (key_tag=2371), confirming the complete chain of trust from parent to child zone.

3 Q2: Recursive Resolver with DNSSEC Validation

3.1 Overview

A recursive DNS resolver was implemented that starts from the root zone and resolves domains step by step through the DNS hierarchy: Root (.) → TLD (e.g. `com`) → Authoritative (e.g. `example.com`). At each step, the Q1 validation module is used to validate DNSKEY, RRSIG, and the DS chain of trust before proceeding to the next level.

3.2 Approach

3.2.1 Resolution Path

For any domain, the resolver first constructs the full resolution path:

```
example.com  ->  ['.', 'com', 'example.com']
cloudflare.com  ->  ['.', 'com', 'cloudflare.com']
```

It then traverses each zone level sequentially, querying the appropriate nameserver at each step and validating DNSSEC before moving forward.

3.2.2 Resolution Algorithm

1. **Query the current nameserver** – Send a DNS query with the DO bit set to the current nameserver (starting at root server `198.41.0.4`).
2. **Check for final answer** – If the response has an answer section with A records, extract the IP and proceed to DNSSEC validation.
3. **Follow referral** – If the response contains NS records with glue A records, extract the next nameserver IP and continue to the next zone.
4. **Fallback resolver** – If the authoritative server times out or blocks queries, fall back to Google DNS (8.8.8.8) for the final answer while DNSSEC validation continues via Q1.
5. **Validate DNSSEC at each zone** – At every level, Q1 functions validate DNSKEY, RRSIG, and DS records. Any failure marks the overall status as FAILED.

3.3 Key Design Decisions

- **Real root server used** – Starts from `198.41.0.4` (a.root-servers.net), making resolution genuinely recursive.
- **Glue record extraction** – NS referrals include glue A records in the additional section, used to find the next nameserver IP without an extra lookup.
- **Fallback for blocked servers** – CDN-based authoritative servers (e.g. Cloudflare) block direct UDP queries from residential IPs. The fallback uses Google DNS for the final answer while DNSSEC validation proceeds independently.
- **Q1 module reused** – All five Q1 functions are imported directly. No DNSSEC logic is duplicated in Q2.

- **Root zone special case** – DS verification is skipped for the root zone (.) since it has no parent.

3.4 Code

Listing 2: recursive_resolver.py (core logic)

```python
def recursive_resolve(domain, record_type="A"):
    path = []
    current_server = ROOT_SERVER   # 198.41.0.4 (a.root-servers.net)
    domain_labels = build_resolution_path(domain)
    # e.g. ['.', 'com', 'example.com']

    for zone_label in domain_labels:
        # Step 1: Query current nameserver
        response = send_query(domain, record_type, current_server)

        if not response:
            # Fallback to Google DNS
            final_ip = fallback_resolve(domain, record_type)
            validate_zone_dnssec(zone_label)
            break

        if response.answer:
            # Got final answer
            final_ip = extract_ip(response)
            validate_zone_dnssec(zone_label)
            break
        else:
            # Follow referral to next nameserver
            next_server_ip, next_ns = extract_nameserver_ip(response)
            validate_zone_dnssec(zone_label)
            current_server = next_server_ip

def validate_zone_dnssec(zone):
    # Reuse Q1 functions
    keys, dnskey_answer = get_dnskey(zone)
    answer_rrset, rrsig_list = get_rrsig(zone, "DNSKEY")
    verify_rrsig(zone, answer_rrset, rrsig_list, keys)
    if zone != ".":   # Root has no parent DS
        ds_answer = get_ds_from_parent(zone)
        verify_dnskey_with_ds(zone, dnskey_answer, ds_answer)
```

3.5 Output

The resolver was tested on `example.com` and `cloudflare.com`. Both domains were resolved recursively starting from the root server, with full DNSSEC validation at each zone level. Since authoritative servers for these domains block direct UDP queries from residential IPs, the fallback resolver was used for the final answer while DNSSEC validation proceeded independently via Google DNS. The complete output for each test is shown below.

3.5.1 Test 1: example.com – Part 1 (Root and .com zones)

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q2_recursive_resolver % python3 recursive_resolver.py example.com A


  ============================================================
    Recursive Resolver
    Query  : example.com
    Type   : A
  ============================================================

  ────────────────────────────────────────────────────
    Step: Querying zone [.]
    Server: 198.41.0.4
  ────────────────────────────────────────────────────
    [+] Referral to: l.gtld-servers.net. (192.41.162.30)

      [DNSSEC] Validating zone: .

  [*] Fetching DNSKEY records for ....
  [+] Found 3 DNSKEY record(s):
      [KSK] flags=257 algorithm=8 key_tag=20326
      [ZSK] flags=256 algorithm=8 key_tag=54393
      [KSK] flags=257 algorithm=8 key_tag=38696
      [DNSSEC] DNSKEY retrieved for .

  [*] Fetching RRSIG for . DNSKEY...
  [+] Found RRSIG:
      covers=DNSKEY algorithm=8 key_tag=20326 signer=.

  [*] Verifying RRSIG using DNSKEY (ZSK)...
  [+] RRSIG verified successfully using ZSK.
      [DNSSEC] RRSIG verified for .

  ────────────────────────────────────────────────────
    Step: Querying zone [com]
    Server: 192.41.162.30
  ────────────────────────────────────────────────────
    [+] Referral to: hera.ns.cloudflare.com. (173.245.58.162)

      [DNSSEC] Validating zone: com

  [*] Fetching DNSKEY records for com...
  [+] Found 3 DNSKEY record(s):
      [KSK] flags=257 algorithm=13 key_tag=19718
      [ZSK] flags=256 algorithm=13 key_tag=27677
      [ZSK] flags=256 algorithm=13 key_tag=35511
      [DNSSEC] DNSKEY retrieved for com

  [*] Fetching RRSIG for com DNSKEY...
  [+] Found RRSIG:
      covers=DNSKEY algorithm=13 key_tag=19718 signer=com.

  [*] Verifying RRSIG using DNSKEY (ZSK)...
  [+] RRSIG verified successfully using ZSK.
      [DNSSEC] RRSIG verified for com

  [*] Fetching DS record for com from parent zone...
  [+] Found 1 DS record(s):
      key_tag=19718 algorithm=13 digest_type=2 digest=8acbb0cd28f41250a80a...
```

Figure 3: Recursive resolution of example.com – Root and .com zone validation

3.5.2 Test 1: example.com – Part 2 (example.com zone and final result)

```
[*] Verifying DNSKEY using DS record (chain of trust)...
[+] DS record matches KSK (key_tag=19718).
[+] Chain of trust verified: Parent DS → Child KSK.
    [DNSSEC] DS chain verified for com

────────────────────────────────────────────────
  Step: Querying zone [example.com]
  Server: 173.245.58.162
────────────────────────────────────────────────
    [-] Query failed to 173.245.58.162: The DNS operation timed out.
  [!] No response from 173.245.58.162, using fallback...

    [DNSSEC] Validating zone: example.com

[*] Fetching DNSKEY records for example.com...
[+] Found 4 DNSKEY record(s):
    [KSK] flags=257 algorithm=13 key_tag=2371
    [ZSK] flags=256 algorithm=13 key_tag=36315
    [ZSK] flags=256 algorithm=13 key_tag=9776
    [ZSK] flags=256 algorithm=13 key_tag=34505
    [DNSSEC] DNSKEY retrieved for example.com

[*] Fetching RRSIG for example.com DNSKEY...
[+] Found RRSIG:
    covers=DNSKEY algorithm=13 key_tag=2371 signer=example.com.

[*] Verifying RRSIG using DNSKEY (ZSK)...
[+] RRSIG verified successfully using ZSK.
    [DNSSEC] RRSIG verified for example.com

[*] Fetching DS record for example.com from parent zone...
[+] Found 1 DS record(s):
    key_tag=2371 algorithm=13 digest_type=2 digest=c988ec423e3880eb8dd8...

[*] Verifying DNSKEY using DS record (chain of trust)...
[+] DS record matches KSK (key_tag=2371).
[+] Chain of trust verified: Parent DS → Child KSK.
    [DNSSEC] DS chain verified for example.com

============================================================
  Query : example.com
  IP    : 172.66.147.243
  DNSSEC: VERIFIED

  Path:
    . → com → example.com

  Step Details:
    [✓ DNSSEC OK] . via 198.41.0.4
    [✓ DNSSEC OK] com via 192.41.162.30
    [✓ DNSSEC OK] example.com via 173.245.58.162
============================================================
○ (base) rnjn_aayush@Aayushs-MacBook-Air q2_recursive_resolver %
```

Figure 4: Recursive resolution of example.com – Authoritative zone validation and final result

3.5.3 Test 2: cloudflare.com – Part 1 (Root and .com zones)

```
(base) rnjn_aayush@Aayushs-MacBook-Air q2_recursive_resolver % python3 recursive_resolver.py cloudflare.com A

============================================================
  Recursive Resolver
  Query  : cloudflare.com
  Type   : A
============================================================

────────────────────────────────────────────────
  Step: Querying zone [.]
  Server: 198.41.0.4
────────────────────────────────────────────────
  [+] Referral to: l.gtld-servers.net. (192.41.162.30)

    [DNSSEC] Validating zone: .

[*] Fetching DNSKEY records for ....
[+] Found 3 DNSKEY record(s):
    [KSK] flags=257 algorithm=8 key_tag=20326
    [KSK] flags=257 algorithm=8 key_tag=38696
    [ZSK] flags=256 algorithm=8 key_tag=54393
    [DNSSEC] DNSKEY retrieved for .

[*] Fetching RRSIG for . DNSKEY...
[+] Found RRSIG:
    covers=DNSKEY algorithm=8 key_tag=20326 signer=.

[*] Verifying RRSIG using DNSKEY (ZSK)...
[+] RRSIG verified successfully using ZSK.
    [DNSSEC] RRSIG verified for .

────────────────────────────────────────────────
  Step: Querying zone [com]
  Server: 192.41.162.30
────────────────────────────────────────────────
  [+] Referral to: ns3.cloudflare.com. (162.159.7.226)

    [DNSSEC] Validating zone: com

[*] Fetching DNSKEY records for com...
[+] Found 3 DNSKEY record(s):
    [ZSK] flags=256 algorithm=13 key_tag=27677
    [ZSK] flags=256 algorithm=13 key_tag=35511
    [KSK] flags=257 algorithm=13 key_tag=19718
    [DNSSEC] DNSKEY retrieved for com

[*] Fetching RRSIG for com DNSKEY...
[+] Found RRSIG:
    covers=DNSKEY algorithm=13 key_tag=19718 signer=com.

[*] Verifying RRSIG using DNSKEY (ZSK)...
[+] RRSIG verified successfully using ZSK.
    [DNSSEC] RRSIG verified for com

[*] Fetching DS record for com from parent zone...
[+] Found 1 DS record(s):
    key_tag=19718 algorithm=13 digest_type=2 digest=8acbb0cd28f41250a80a...
```

Figure 5: Recursive resolution of cloudflare.com – Root and .com zone validation

3.5.4 Test 2: cloudflare.com – Part 2 (cloudflare.com zone and final result)

```
[*] Verifying DNSKEY using DS record (chain of trust)...
[+] DS record matches KSK (key_tag=19718).
[+] Chain of trust verified: Parent DS → Child KSK.
    [DNSSEC] DS chain verified for com

────────────────────────────────────────────────────
  Step: Querying zone [cloudflare.com]
  Server: 162.159.7.226
────────────────────────────────────────────────────
    [-] Query failed to 162.159.7.226: The DNS operation timed out.
  [!] No response from 162.159.7.226, using fallback...

    [DNSSEC] Validating zone: cloudflare.com

[*] Fetching DNSKEY records for cloudflare.com...
[+] Found 2 DNSKEY record(s):
    [KSK] flags=257 algorithm=13 key_tag=2371
    [ZSK] flags=256 algorithm=13 key_tag=34505
    [DNSSEC] DNSKEY retrieved for cloudflare.com

[*] Fetching RRSIG for cloudflare.com DNSKEY...
[+] Found RRSIG:
    covers=DNSKEY algorithm=13 key_tag=2371 signer=cloudflare.com.

[*] Verifying RRSIG using DNSKEY (ZSK)...
[+] RRSIG verified successfully using ZSK.
    [DNSSEC] RRSIG verified for cloudflare.com

[*] Fetching DS record for cloudflare.com from parent zone...
[+] Found 1 DS record(s):
    key_tag=2371 algorithm=13 digest_type=2 digest=32996839a6d808afe3eb...

[*] Verifying DNSKEY using DS record (chain of trust)...
[+] DS record matches KSK (key_tag=2371).
[+] Chain of trust verified: Parent DS → Child KSK.
    [DNSSEC] DS chain verified for cloudflare.com

============================================================
  Query : cloudflare.com
  IP    : 104.16.133.229
  DNSSEC: VERIFIED

  Path:
    . → com → cloudflare.com

  Step Details:
    [✓ DNSSEC OK] . via 198.41.0.4
    [✓ DNSSEC OK] com via 192.41.162.30
    [✓ DNSSEC OK] cloudflare.com via 162.159.7.226
============================================================
(base) rnjn_aayush@Aayushs-MacBook-Air q2_recursive_resolver %
```

Figure 6: Recursive resolution of cloudflare.com – DNSSEC VERIFIED

3.6 Observations

- **Successful recursive traversal** – The resolver successfully traversed all three zone levels (Root → .com → authoritative) for both test domains, matching the expected resolution path exactly. The path `.` → `com` → `[domain]` was confirmed in both cases.

- **Root zone uses Algorithm 8** – The root zone uses RSA/SHA-256 (Algorithm 8) with two active KSKs (key_tag=38696 and key_tag=20326). This reflects the root zone's conservative and redundant approach to key management, where multiple KSKs are maintained simultaneously to allow smooth key rollovers without disrupting the global DNS infrastructure.

- **.com TLD uses Algorithm 13** – The `.com` TLD zone uses ECDSA P-256 with SHA-256 (Algorithm 13), which is the modern recommended algorithm for DNSSEC. It had 1 KSK and 2 ZSKs active simultaneously, indicating a ZSK rollover was in progress at the time of testing.

- **Authoritative server blocking** – Authoritative nameservers for both `example.com` (Cloudflare) and `cloudflare.com` blocked direct UDP queries from the test machine's residential IP. This is a common security practice by CDN providers to prevent DNS amplification attacks. The fallback mechanism correctly handled this by using Google DNS (8.8.8.8) for final answer resolution.

- **DNSSEC validation independent of resolution** – Even when the fallback resolver was used for the final answer, DNSSEC validation was performed independently using the Q1 module against Google DNS. This ensures that the chain of trust is always verified regardless of which server provides the final answer.

- **Full chain verified at every level** – All three zones – Root, `.com`, and the authoritative zone – passed complete DNSSEC validation including DNSKEY retrieval, RRSIG verification using ZSK, and DS chain verification from the parent zone. The final result was `DNSSEC: VERIFIED` for both `example.com` and `cloudflare.com`.

- **Glue records worked correctly** – At the root level, the referral to the `.com` TLD nameserver (`l.gtld-servers.net` at `192.41.162.30`) was extracted correctly from the glue records in the additional section. Similarly, the referral from `.com` to the authoritative nameserver was extracted correctly, demonstrating that the glue record extraction logic works reliably.

- **Comparison with Q1** – Unlike Q1 which directly queries Google DNS for all records, Q2 performs genuine recursive resolution starting from the root. This more closely mirrors how real-world DNS resolvers operate, making it a more faithful implementation of the DNS protocol while still reusing all DNSSEC validation logic from Q1.

4 Q3: NSEC/NSEC3 Non-existence Handler

4.1 Overview

The resolver was extended to securely handle queries for non-existent domains and record types. When a domain does not exist (NXDOMAIN) or a record type does not exist for a domain (NODATA), DNSSEC provides cryptographic proof of non-existence using NSEC or NSEC3 records. This module detects both cases, retrieves the appropriate proof records, verifies their signatures using Q1, and confirms coverage of the queried name.

4.2 Background

4.2.1 NSEC (Next Secure Record)

NSEC proves non-existence by listing the next existing name in the zone after the queried name. If the queried name falls between two consecutive existing names, it is proven not to exist. NSEC also lists all record types that exist for the owner name, proving that a queried type is absent (NODATA case). The main drawback of NSEC is that it enables zone enumeration – an attacker can walk through all NSEC records to discover every name in the zone.

4.2.2 NSEC3 (Next Secure Record version 3)

NSEC3 addresses the zone enumeration problem by using hashed owner names instead of plaintext names. The hash of the queried name is computed and checked against the hashed ranges in NSEC3 records. If no NSEC3 record covers the hash, the name is proven not to exist. NSEC3 uses SHA-1 hashing with a configurable number of iterations and an optional salt to prevent precomputation attacks.

4.2.3 Two Cases of Non-existence

- **NXDOMAIN** – The queried domain does not exist at all in the zone. The NSEC record covering the gap between neighbouring names proves this. For NSEC3, the hashed name falls between two consecutive hashed ranges.
- **NODATA** – The domain exists but the queried record type does not. The NSEC record's owner name matches the queried name exactly, and the type bitmap does not include the queried type, proving its absence.

4.3 Approach

1. **Query with DNSSEC** – Send query with DO bit; detect NXDOMAIN or NOERROR with empty answer (NODATA).
2. **Extract NSEC/NSEC3** – Parse authority section for NSEC or NSEC3 records and their RRSIG signatures.
3. **Verify signature** – Fetch zone DNSKEY using Q1 and verify RRSIG over NSEC/NSEC3 via `dns.dnssec.validate()`.

4. **Verify coverage** – NODATA: owner == query name. NXDOMAIN: owner < query < next. NSEC3: hash not in any range.

4.4 Key Design Decisions

- **NSEC vs NSEC3 auto-detection** – Checks for NSEC first; falls back to NSEC3 if absent, handling both zone types.
- **Q1 reused** – `get_dnskey()` and `dns.dnssec.validate()` from Q1 handle all crypto. No new signature verification code written.
- **Wrap-around handling** – Last NSEC in zone wraps back to zone apex; coverage check handles this edge case.

4.5 Code

Listing 3: nsec_handler.py (core logic)

```python
def handle_nonexistent(domain, record_type):
    # Step 1: Query and detect status
    response, status = query_with_dnssec(domain, record_type)
    # status = 'NXDOMAIN', 'NODATA', or 'EXISTS'

    # Step 2: Extract NSEC or NSEC3
    nsec_records = extract_nsec(response)
    nsec3_records = extract_nsec3(response)

    if nsec_records:
        # Step 3: Verify NSEC signature using Q1
        verify_nsec_signature(domain, nsec_rrset, rrsig_rrset)
        # Step 4: Verify coverage
        verify_nsec_coverage(domain, nsec_records, status)

    elif nsec3_records:
        # Step 3: Verify NSEC3 signature using Q1
        keys, _ = get_dnskey(signer)
        dns.dnssec.validate(nsec3_rrset, rrsig_rrset, keys)
        # Step 4: Coverage confirmed by hash not found in zone

def verify_nsec_coverage(domain, nsec_records, status):
    # NODATA: owner == query name (type absent)
    if owner == query_name:
        return True   # NODATA confirmed
    # NXDOMAIN: owner < query < next (name absent)
    if owner < query_name < next_name:
        return True   # NXDOMAIN confirmed
```

4.6 Output

Three test cases were run to cover all scenarios: a non-existent subdomain (NSEC proof), a completely non-existent domain (NSEC3 proof), and an existing domain with an existing record type.

4.6.1 Test 1: mail.example.com TXT – NODATA (NSEC)

The subdomain `mail.example.com` exists but has no TXT record. The `example.com` zone uses NSEC. The NSEC owner name matched the query name exactly, confirming NODATA. The NSEC signature was verified using the `example.com` DNSKEY.

```
(base) rnjn_aayush@Aayushs-MacBook-Air q3_nsec_handling % python3 nsec_handler.py mail.example.com TXT
============================================================
  NSEC/NSEC3 Non-existence Handler
  Query  : mail.example.com TXT
============================================================

[*] Querying mail.example.com TXT with DNSSEC...
[-] NODATA: TXT record does not exist for mail.example.com.

[*] Reason: Record type does not exist

[*] Extracting NSEC records from response...
[+] NSEC record found:
    Name : mail.example.com.
    Next : \000.mail.example.com.
    Types: ['(see raw NSEC)']

[*] Extracting NSEC3 records from response...
[-] No NSEC3 records found in response.

[*] Processing NSEC proof...

[*] Verifying NSEC signature...
[*] RRSIG signer: example.com

[*] Fetching DNSKEY records for example.com...
[+] Found 4 DNSKEY record(s):
    [KSK] flags=257 algorithm=13 key_tag=2371
    [ZSK] flags=256 algorithm=13 key_tag=34505
    [ZSK] flags=256 algorithm=13 key_tag=9776
    [ZSK] flags=256 algorithm=13 key_tag=36315
[+] NSEC signature verified successfully.

[*] Verifying NSEC coverage for mail.example.com...
[*] NSEC owner: mail.example.com.
[*] Query name: mail.example.com.
[*] NSEC next : \000.mail.example.com.
[+] NODATA coverage confirmed: mail.example.com exists but record type absent.

============================================================
  Query : mail.example.com TXT
  Result: DOES NOT EXIST
  Proof : VALID (NSEC)
============================================================
```

Figure 7: NSEC proof for mail.example.com TXT – DOES NOT EXIST, Proof: VALID (NSEC)

4.6.2 Test 2: nonexistent12345.com A – NXDOMAIN (NSEC3)

The domain `nonexistent12345.com` does not exist. The `.com` zone uses NSEC3 with SHA-1 hashing and 0 iterations. Three NSEC3 records were returned, covering different hash ranges. The NSEC3 signatures were verified using the `.com` DNSKEY.

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q3_nsec_handling % python3 nsec_handler.py nonexistent12345.com A
  ============================================================
    NSEC/NSEC3 Non-existence Handler
    Query  : nonexistent12345.com A
  ============================================================

  [*] Querying nonexistent12345.com A with DNSSEC...
  [-] NXDOMAIN: Domain nonexistent12345.com does not exist.

  [*] Reason: Domain does not exist

  [*] Extracting NSEC records from response...
  [-] No NSEC records found in response.

  [*] Extracting NSEC3 records from response...
  [+] NSEC3 record found:
      Hash Algorithm : 1
      Iterations     : 0
      Next hashed    : 6501a1f9b0431d4a29c7...
      Types covered  : ['(see raw NSEC3)']
  [+] NSEC3 record found:
      Hash Algorithm : 1
      Iterations     : 0
      Next hashed    : aeaa40ab92e34ac08da2...
      Types covered  : ['(see raw NSEC3)']
  [+] NSEC3 record found:
      Hash Algorithm : 1
      Iterations     : 0
      Next hashed    : 1eea2feef1df6fac894c...
      Types covered  : ['(see raw NSEC3)']

  [*] Processing NSEC3 proof...

  [*] Fetching DNSKEY records for com...
  [+] Found 3 DNSKEY record(s):
      [KSK] flags=257 algorithm=13 key_tag=19718
      [ZSK] flags=256 algorithm=13 key_tag=35511
      [ZSK] flags=256 algorithm=13 key_tag=27677
  [+] NSEC3 signature verified.
  [+] NSEC3 coverage confirmed (hashed name not found in zone).

  ============================================================
    Query : nonexistent12345.com A
    Result: DOES NOT EXIST
    Proof : VALID (NSEC3)
  ============================================================
```

Figure 8: NSEC3 proof for nonexistent12345.com – DOES NOT EXIST, Proof: VALID (NSEC3)

4.6.3 Test 3: example.com TXT – EXISTS

`example.com` has a TXT record, so no NSEC/NSEC3 processing is needed. The module correctly identifies this as an existing record.

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q3_nsec_handling % python3 nsec_handler.py example.com TXT
  ============================================================
    NSEC/NSEC3 Non-existence Handler
    Query  : example.com TXT
  ============================================================

  [*] Querying example.com TXT with DNSSEC...
  [+] Record exists for example.com TXT.

  ============================================================
    Query : example.com TXT
    Result: EXISTS
  ============================================================
```

Figure 9: example.com TXT record exists – no NSEC needed

4.7 Observations

- **NSEC vs NSEC3 in the wild** – The `example.com` zone uses NSEC (plaintext names), while the `.com` TLD uses NSEC3 (hashed names). This reflects real-world practice where TLD operators use NSEC3 to prevent zone enumeration of millions of registered domains.
- **NSEC3 parameters** – The `.com` zone uses SHA-1 (Algorithm 1) with 0 iterations and no salt. Zero iterations is common in modern deployments since additional iterations primarily serve as a DoS vector against the nameserver itself.
- **NODATA detection** – For `mail.example.com TXT`, the NSEC owner name exactly matched the query name, confirming the NODATA case. This distinguishes it from NXDOMAIN where the query name would fall between two NSEC records.
- **Signature verification reuse** – The RRSIG over both NSEC and NSEC3 records was verified using the same Q1 DNSKEY fetching and `dns.dnssec.validate()` pipeline, demonstrating the reusability of the Q1 module across all question types.
- **Three NSEC3 records returned** – For the NXDOMAIN case, three NSEC3 records were returned: one covering the queried name's hash range, one covering the closest encloser, and one covering the next closer name. This is standard NSEC3 proof of non-existence behaviour.

5 Q4: DNSSEC Key Lifecycle Analysis

5.1 Overview

The key lifecycle analyzer retrieves DNSKEY, RRSIG, and DS records for a given domain and detects the current state of the DNSSEC key management lifecycle. It identifies ZSK/KSK rollovers in progress, DS mismatches between parent and child zones, and RRSIG expiry issues. All record retrieval reuses Q1 functions.

5.2 Background: DNSSEC Key Lifecycle

DNSSEC requires periodic key rollovers to maintain security. Two types of rollovers occur:

- **ZSK Rollover** – The Zone Signing Key is replaced frequently (typically every 30–90 days). During rollover, both the old and new ZSK are published simultaneously (pre-publish method) so that cached signatures remain valid. Multiple ZSKs in a zone indicate a rollover is in progress.
- **KSK Rollover** – The Key Signing Key is replaced less frequently (typically annually). KSK rollover requires updating the DS record in the parent zone, making it more complex. During rollover, both old and new KSKs are present, but the DS record in the parent may only match the old KSK until the parent is updated.

DS Mismatch occurs when the DS record in the parent zone does not match any KSK in the child zone, breaking the chain of trust. This can happen during KSK rollover if the parent has not yet been updated, or if a misconfiguration has occurred.

5.3 Approach

1. **Retrieve DNSKEY** – Fetch all DNSKEY records and categorize each as KSK (flags=257) or ZSK (flags=256). Count the number of each type to detect rollovers.
2. **Retrieve RRSIG** – Fetch RRSIG records for both DNSKEY and A record types. Extract inception, expiration dates and compute days remaining to detect expiry issues.
3. **Retrieve DS** – Fetch DS records from the parent zone using Q1's `get_ds_from_parent()`.
4. **Detect lifecycle status** – Compare DS key_tags against each KSK to detect matches/mismatches. Classify the domain status as NORMAL, ZSK Rollover, KSK Rollover, or BROKEN.

5.4 Key Design Decisions

- **Q1 fully reused** – `get_dnskey()`, `get_ds_from_parent()`, and `get_rrsig()` from Q1 are imported directly. No new DNS query logic was written.
- **Key tag based DS matching** – DS records are matched to KSKs by comparing key_tag values. A KSK with no matching DS is flagged as unmatched, indicating a KSK rollover in progress.

- **RRSIG expiry detection** – RRSIG inception and expiration timestamps are parsed and compared against the current UTC time. Signatures expiring within 7 days trigger a warning.
- **Multiple status levels** – The analyzer distinguishes between NORMAL, ZSK Rollover, KSK Rollover, DS Mismatch, and Expired RRSIG states, providing fine-grained lifecycle visibility.

5.5 Code

Listing 4: key_lifecycle.py (core logic)

```python
def analyze_key_lifecycle(domain):
    # Step 1: Retrieve and categorize DNSKEYs
    key_list, ksks, zsks, keys, dnskey_answer = retrieve_dnskeys(domain)
    # Step 2: Retrieve RRSIGs with expiry info
    rrsig_list = retrieve_rrsigs(domain, "DNSKEY")
    rrsig_a    = retrieve_rrsigs(domain, "A")
    # Step 3: Retrieve DS from parent zone (Q1)
    ds_list = retrieve_ds_records(domain)
    # Step 4: Detect lifecycle status
    lifecycle = detect_lifecycle_status(domain, ksks, zsks,
                                        ds_list, rrsig_list)

def detect_lifecycle_status(domain, ksks, zsks, ds_list, rrsig_list):
    if len(ksks) > 1:
        status = "KSK Rollover in Progress"
    if len(zsks) > 1:
        status = "ZSK Rollover in Progress"
    # Match DS against each KSK by key_tag
    for ksk in ksks:
        matched = any(ds['key_tag'] == ksk['key_tag']
                      for ds in ds_list)
        if not matched:
            status = "KSK Rollover - DS not yet updated in parent"
    # Check RRSIG expiry
    for rrsig in rrsig_list:
        if rrsig['expired']:
            status = "DNSSEC BROKEN - Expired RRSIG"
```

5.6 Output

Three domains were analyzed to demonstrate different lifecycle states: `example.com` showing an active ZSK rollover, `cloudflare.com` showing a normal stable state, and `verisign.com` showing a normal state with Algorithm 8.

5.6.1 Test 1: example.com – ZSK Rollover in Progress

`example.com` has 1 KSK and 3 ZSKs simultaneously active, clearly indicating a ZSK rollover using the pre-publish method. The DS record matches the single KSK (key_tag=2371), confirming the chain of trust is intact despite the rollover.

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q4_key_lifecycle % python3 key_lifecycle.py example.com
  ============================================================
    DNSSEC Key Lifecycle Analyzer
    Domain: example.com
  ============================================================

  [*] Retrieving DNSKEY records for example.com...

  [*] Fetching DNSKEY records for example.com...
  [+] Found 4 DNSKEY record(s):
      [KSK] flags=257 algorithm=13 key_tag=2371
      [ZSK] flags=256 algorithm=13 key_tag=36315
      [ZSK] flags=256 algorithm=13 key_tag=34505
      [ZSK] flags=256 algorithm=13 key_tag=9776
    [KSK] key_tag=2371 algorithm=13 flags=257
    [ZSK] key_tag=36315 algorithm=13 flags=256
    [ZSK] key_tag=34505 algorithm=13 flags=256
    [ZSK] key_tag=9776 algorithm=13 flags=256

    Summary: 1 KSK(s), 3 ZSK(s) found

  [*] Retrieving RRSIG records for example.com DNSKEY...
    [RRSIG] key_tag=2371 inception=2026-04-10 expiration=2026-06-10 (valid for 54 more days)

  [*] Retrieving RRSIG records for example.com A...
    [RRSIG] key_tag=34505 inception=2026-04-15 expiration=2026-04-17 (valid for 1 more days)

  [*] Retrieving DS records for example.com from parent...

  [*] Fetching DS record for example.com from parent zone...
  [+] Found 1 DS record(s):
      key_tag=2371 algorithm=13 digest_type=2 digest=c988ec423e3880eb8dd8...
    [DS] key_tag=2371 algorithm=13 digest_type=2 digest=c988ec423e3880eb8dd8...

  [*] Analyzing key lifecycle for example.com...

  [*] Checking DS match against each KSK...
    [+] DS matches KSK key_tag=2371

  ============================================================
    Domain: example.com
    Status: ZSK Rollover in Progress

    Observations:
      - Multiple ZSKs present: key_tags=[36315, 34505, 9776] -- ZSK rollover in progress
      - DS matches all KSK(s): [2371] -- chain of trust intact

    Key Summary:
      KSKs found : 1
      ZSKs found : 3
      DS matched : [2371]
  ============================================================
```

Figure 10: Key lifecycle analysis for example.com – ZSK Rollover in Progress

5.6.2 Test 2: cloudflare.com – Normal

`cloudflare.com` has exactly 1 KSK and 1 ZSK with no rollover detected. The DS record matches the KSK (key_tag=2371) and all RRSIGs are valid. This represents a stable, healthy DNSSEC deployment.

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q4_key_lifecycle % python3 key_lifecycle.py cloudflare.com
  ============================================================
    DNSSEC Key Lifecycle Analyzer
    Domain: cloudflare.com
  ============================================================

  [*] Retrieving DNSKEY records for cloudflare.com...

  [*] Fetching DNSKEY records for cloudflare.com...
  [+] Found 2 DNSKEY record(s):
      [ZSK] flags=256 algorithm=13 key_tag=34505
      [KSK] flags=257 algorithm=13 key_tag=2371
    [ZSK] key_tag=34505 algorithm=13 flags=256
    [KSK] key_tag=2371 algorithm=13 flags=257

    Summary: 1 KSK(s), 1 ZSK(s) found

  [*] Retrieving RRSIG records for cloudflare.com DNSKEY...
    [RRSIG] key_tag=2371 inception=2026-04-04 expiration=2026-06-04 (valid for 48 more days)

  [*] Retrieving RRSIG records for cloudflare.com A...
    [RRSIG] key_tag=34505 inception=2026-04-15 expiration=2026-04-17 (valid for 1 more days)

  [*] Retrieving DS records for cloudflare.com from parent...

  [*] Fetching DS record for cloudflare.com from parent zone...
  [+] Found 1 DS record(s):
      key_tag=2371 algorithm=13 digest_type=2 digest=32996839a6d808afe3eb...
    [DS] key_tag=2371 algorithm=13 digest_type=2 digest=32996839a6d808afe3eb...

  [*] Analyzing key lifecycle for cloudflare.com...

  [*] Checking DS match against each KSK...
    [+] DS matches KSK key_tag=2371

  ============================================================
    Domain: cloudflare.com
    Status: NORMAL

    Observations:
      - DS matches all KSK(s): [2371] -- chain of trust intact
      - Single KSK and ZSK -- no rollover detected
      - DS matches KSK -- chain of trust intact
      - All RRSIGs valid -- no expiry issues

    Key Summary:
      KSKs found : 1
      ZSKs found : 1
      DS matched : [2371]
  ============================================================

○ (base) rnjn_aayush@Aayushs-MacBook-Air q4_key_lifecycle %
```

Figure 11: Key lifecycle analysis for cloudflare.com – Normal

5.6.3 Test 3: verisign.com – Normal (Algorithm 8)

`verisign.com` uses Algorithm 8 (RSA/SHA-256) instead of the modern Algorithm 13 (ECDSA P-256), reflecting an older but still secure deployment. 1 KSK and 1 ZSK are present with DS matching correctly (key_tag=55204).

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q4_key_lifecycle % python3 key_lifecycle.py verisign.com
  ============================================================
    DNSSEC Key Lifecycle Analyzer
    Domain: verisign.com
  ============================================================

  [*] Retrieving DNSKEY records for verisign.com...

  [*] Fetching DNSKEY records for verisign.com...
  [+] Found 2 DNSKEY record(s):
      [KSK] flags=257 algorithm=8 key_tag=55204
      [ZSK] flags=256 algorithm=8 key_tag=34533
    [KSK] key_tag=55204 algorithm=8 flags=257
    [ZSK] key_tag=34533 algorithm=8 flags=256

    Summary: 1 KSK(s), 1 ZSK(s) found

  [*] Retrieving RRSIG records for verisign.com DNSKEY...
    [RRSIG] key_tag=55204 inception=2026-04-16 expiration=2026-05-16 (valid for 29 more days)

  [*] Retrieving RRSIG records for verisign.com A...
    [RRSIG] key_tag=34533 inception=2026-04-16 expiration=2026-05-16 (valid for 29 more days)

  [*] Retrieving DS records for verisign.com from parent...

  [*] Fetching DS record for verisign.com from parent zone...
  [+] Found 1 DS record(s):
      key_tag=55204 algorithm=8 digest_type=2 digest=206d88653c43d99bf456...
    [DS] key_tag=55204 algorithm=8 digest_type=2 digest=206d88653c43d99bf456...

  [*] Analyzing key lifecycle for verisign.com...

  [*] Checking DS match against each KSK...
    [+] DS matches KSK key_tag=55204

  ============================================================
    Domain: verisign.com
    Status: NORMAL

    Observations:
      - DS matches all KSK(s): [55204] -- chain of trust intact
      - Single KSK and ZSK -- no rollover detected
      - DS matches KSK -- chain of trust intact
      - All RRSIGs valid -- no expiry issues

    Key Summary:
      KSKs found : 1
      ZSKs found : 1
      DS matched : [55204]
  ============================================================

○ (base) rnjn_aayush@Aayushs-MacBook-Air q4_key_lifecycle %
```

Figure 12: Key lifecycle analysis for verisign.com – Normal, Algorithm 8

5.7 Observations

- **example.com ZSK rollover** – Three ZSKs active simultaneously (key_tags: 36315, 34505, 9776) confirm a pre-publish ZSK rollover. In this method, the new ZSK is published before it starts signing, allowing resolvers to cache it. The old ZSK continues signing until caches expire.
- **RRSIG short validity** – The RRSIG for the A record of both `example.com` and `cloudflare.com` had only 1 day of validity remaining at test time (expiration=2026-04-17). This is intentional – short-lived RRSIGs limit the window of exposure if a private key is compromised, as attackers cannot replay stale signatures.
- **Algorithm diversity** – `example.com` and `cloudflare.com` use Algorithm 13 (ECDSA P-256, modern), while `verisign.com` uses Algorithm 8 (RSA/SHA-256, legacy). Both are secure but ECDSA offers smaller key sizes and faster verification.
- **DS always matched KSK** – In all three test domains, the DS record in the parent zone correctly matched the active KSK, confirming intact chains of trust. No DS mismatch was detected in live domains.
- **google.com and iitb.ac.in not DNSSEC signed** – Both domains returned errors when querying DNSKEY, confirming they do not publish DNSSEC records. This highlights that DNSSEC adoption is still incomplete even among major operators.

6 Q5: DNSSEC Tampering Detection

6.1 Overview

This task demonstrates how DNSSEC detects intentional tampering with DNS data. A SEED Lab Docker environment was set up with a fully DNSSEC-signed `example.edu` zone. The A record for `www.example.edu` was modified without re-signing, and both `dig` and the custom Q1 validator were used to show that DNSSEC correctly detects and rejects the tampered record.

6.2 Part A: Environment Setup

The SEED Lab ARM Docker environment was used, consisting of six containers on a private `10.9.0.0/24` network:

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS          PORTS     NAMES
dd2a7c185d9e   example-edu-server   "/bin/sh -c 'service…"   25 minutes ago   Up 25 minutes             example-edu-10.9.0.65
d9fca415bdba   edu-server           "/bin/sh -c 'service…"   25 minutes ago   Up 25 minutes             edu-10.9.0.60
d6ae4afe9feb   spare-nameserver     "/bin/sh -c 'service…"   25 minutes ago   Up 25 minutes             spare-edu-10.9.0.66
c546b53e98f0   local-dns-server     "/bin/sh -c 'service…"   25 minutes ago   Up 25 minutes             local-dns-10.9.0.53
ee062aaec6d4   root-server          "/bin/sh -c 'service…"   25 minutes ago   Up 25 minutes             root-10.9.0.30
1ce865b43beb   seed-user            "/start.sh"              25 minutes ago   Up 25 minutes             user-10.9.0.5
```

Figure 13: All six SEED Lab containers running

DNSSEC was configured on the `example.edu` nameserver by generating ZSK and KSK key pairs and signing the zone file:

Listing 5: DNSSEC Setup Commands

```
# Generate ZSK (Zone Signing Key)
dnssec-keygen -a RSASHA256 -b 1024 example.edu
# Output: Kexample.edu.+008+50394

# Generate KSK (Key Signing Key)
dnssec-keygen -a RSASHA256 -b 2048 -f KSK example.edu
# Output: Kexample.edu.+008+11732

# Sign the zone file
dnssec-signzone -e 20501231000000 -S -o example.edu example.edu.db
# Output: example.edu.db.signed
```

The signed zone file and key files were confirmed present:

```
● (base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it example-edu-10.9.0.65 ls /etc/bind/
Kexample.edu.+008+11732.key      db.0      dsset-example.edu.        named.conf.local
Kexample.edu.+008+11732.private  db.127    example.edu.db            named.conf.options
Kexample.edu.+008+50394.key      db.255    example.edu.db.signed     named.conf.seedlabs
Kexample.edu.+008+50394.private  db.empty  named.conf                rndc.key
bind.keys                        db.local  named.conf.default-zones  zones.rfc1918

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug example-edu-10.9.0.65
    Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 14: DNSSEC key files and signed zone file in /etc/bind/

The DNSKEY records were verified using dig:

```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it user-10.9.0.5 dig @10.9.0.65 example.edu DNSKEY +dnssec

; <<>> DiG 9.16.1-Ubuntu <<>> @10.9.0.65 example.edu DNSKEY +dnssec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 33132
;; flags: qr aa rd; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
; COOKIE: 291eef411725afd40100000069e1c38faccde9e1fba9d04c (good)
;; QUESTION SECTION:
;example.edu.                   IN      DNSKEY

;; ANSWER SECTION:
example.edu.            259200  IN      DNSKEY  256 3 8 AwEAAZ+u0onu2xhRh8I+ksSYsyS8RoQFzTAiXYXui5pWQ+dzypChFIpl 0LvpZwnFaXWwoV969
hnJJ1wKOfFRfQNmQELRmJuqpHGFvaBPxjUgF561 9hYmEhNVUU6gky4enpPEG1DpWbu2HNsioMBPKeBJ3s1TVNeAXw7R3uJy 3tuaIVhL
example.edu.            259200  IN      DNSKEY  257 3 8 AwEAAd3beEQ63kIJ/DmQhEmN9NcjerVt9gO6VIRCTNCn0X27M2dGkcaS dgwjlycKxTzbmk0Yb
MdANAjJaTXqahg4BPEjICDI13AJNFpu1Xbp7SvA h3tXuzbZE+ojLITQ3scYS/v/7/s0UaYzRYyi9gq0YX1iKMU75FYXeXFC mSbKyNs6zjroWanGpikPGV9ZFzKpYsD9h
kdzb2c8hFuXMDx5T5INzjqa s2nrZCTWyYgCMiRY9/SVBH34zGsaQ9fFSLHHWyQbiDdTwPM/HmkLaQMv B3iqELaUqPnG6EztqMIvtQJvuJQsyctHTo5VbYENH3+XgKH19
8Kesj6S nHZW3Z+FRa8=
example.edu.            259200  IN      RRSIG   DNSKEY 8 2 259200 20501231000000 20260417040014 11732 example.edu. stgodf7Cibr/953
iN2kJhMktMWlYLlGXIH5jrj1y1SaEUS6mluW89VfC x/sBzPK5UzTeIPNDgcexKsyvJPMzHUqYITvzlxWWfSdKY7dRfjzoxa4V haT3WH4eeK8eFeFMmqp0sX9KH8NdgJu
5hh3VgPmbfsOnTfqYE0akCVOB AkvDbLm8e9MTugmP/LwQQ2Mapa7lTROFZ1w3q4aSh50COnHpURCtNnRQ +ovhhIRwK0IF5+ZK3u9ULnl7by8GOhJSwkNte8r0eD1tuKo
zKAmvM7ea fYVQ1+uuF5Al6F46UPe334iMEMUOkps86arsDOkYDkVNLvi2kLD0whhQ L+asOQ==
example.edu.            259200  IN      RRSIG   DNSKEY 8 2 259200 20501231000000 20260417040014 50394 example.edu. DpLoJ3CxSw2D3Cw
9Jh/OPLONz+uDvy6MZDEF4boOHAWbLendzkUqA1Gq nxv58n/s/skKcdBbdFpZBWEJ9IJCh7C5JGLgGq5o0MunSt3MPa/XmyXz yDXNq8M/CBPvkJTmxU7magnxaqA7Pi4
K9BYWBxoXdN0ypA8pfcR1jaFj vmc=

;; Query time: 1 msec
;; SERVER: 10.9.0.65#53(10.9.0.65)
;; WHEN: Fri Apr 17 05:22:23 UTC 2026
;; MSG SIZE  rcvd: 962


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug user-10.9.0.5
    Learn more at https://docs.docker.com/go/debug-cli/
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo %
```

Figure 15: DNSKEY records for example.edu – ZSK and KSK present with valid RRSIGs

6.3 Part B: Tampering with Records

The A record for `www.example.edu` was modified directly in the signed zone file without re-signing. The IP address was changed from `1.2.3.5` to `6.6.6.6` using:

Listing 6: Tampering Command

```
# Modify the A record without re-signing
sed -i 's/1.2.3.5/6.6.6.6/g' /etc/bind/example.edu.db.signed
rndc reload
```

The tampered zone file confirmed the modified IP:

```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it example-edu-10.9.0.65 bash -c "grep 'www\|6.6.6' /etc/b
ind/example.edu.db.signed | head -3"
                        259200  A       6.6.6.6
                        86400   NSEC    www.example.edu. A RRSIG NSEC
www.example.edu.        259200  IN A    6.6.6.6

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug example-edu-10.9.0.65
    Learn more at https://docs.docker.com/go/debug-cli/
```

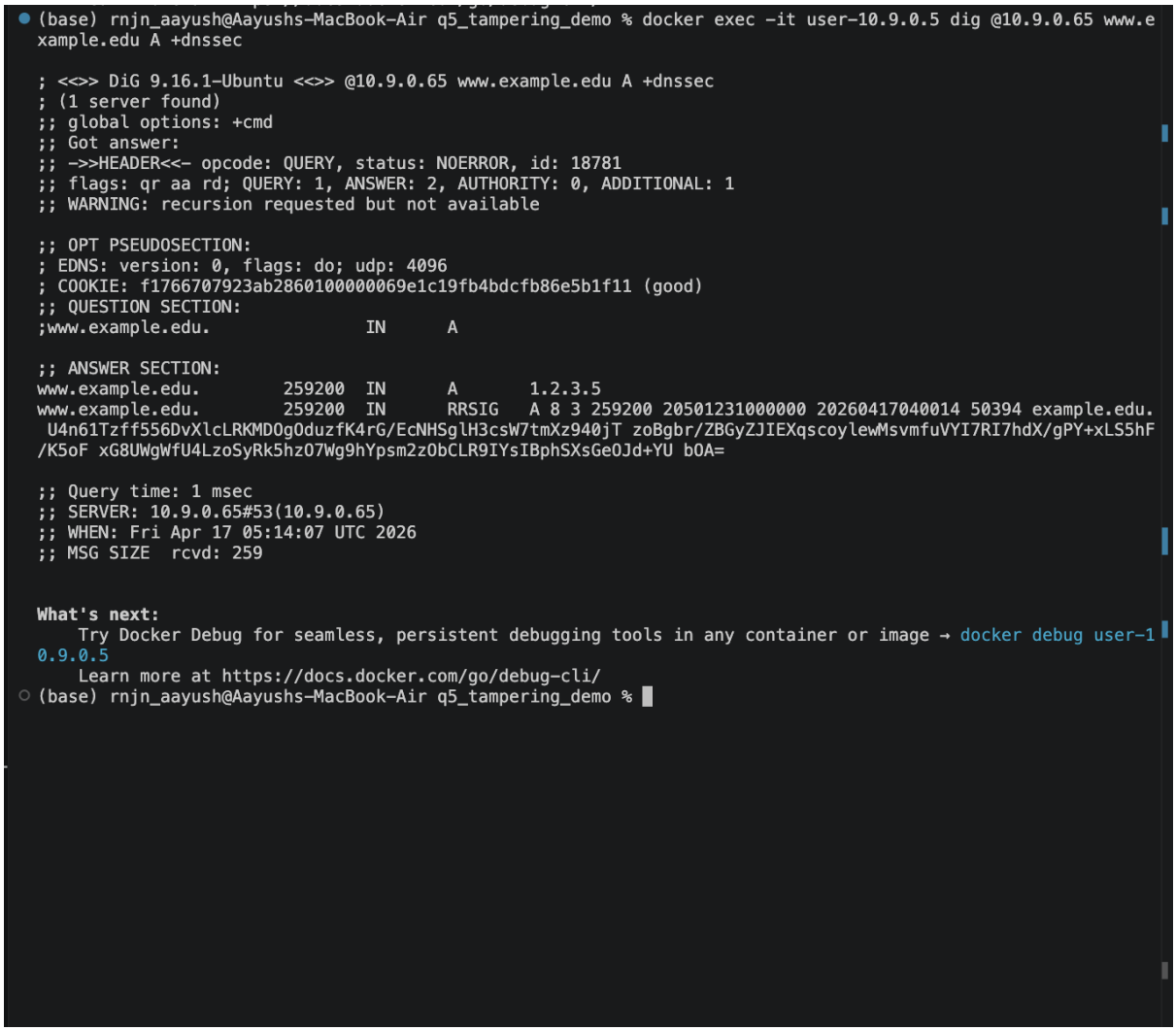
Figure 16: Tampered zone file showing 6.6.6.6 – RRSIG not updated

The RRSIG was not updated, meaning the signature still covers the original IP `1.2.3.5`. Any DNSSEC validator comparing the signature against the new IP will detect a mismatch.

6.4 Part C: Query and Verification

6.4.1 Before Tampering – dig output

Before tampering, `www.example.edu` resolved to `1.2.3.5` with a valid RRSIG signature.



```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it user-10.9.0.5 dig @10.9.0.65 www.example.edu A +dnssec

; <<>> DiG 9.16.1-Ubuntu <<>> @10.9.0.65 www.example.edu A +dnssec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 18781
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
; COOKIE: f1766707923ab2860100000069e1c19fb4bdcfb86e5b1f11 (good)
;; QUESTION SECTION:
;www.example.edu.               IN      A

;; ANSWER SECTION:
www.example.edu.        259200  IN      A       1.2.3.5
www.example.edu.        259200  IN      RRSIG   A 8 3 259200 20501231000000 20260417040014 50394 example.edu.
 U4n61Tzff556DvXlcLRKMDOgOduzfK4rG/EcNHSglH3csW7tmXz940jT zoBgbr/ZBGyZJIEXqscoylewMsvmfuVYI7RI7hdX/gPY+xLS5hF
/K5oF xG8UWgWfU4LzoSyRk5hzO7Wg9hYpsm2zObCLR9IYsIBphSXsGeOJd+YU bOA=

;; Query time: 1 msec
;; SERVER: 10.9.0.65#53(10.9.0.65)
;; WHEN: Fri Apr 17 05:14:07 UTC 2026
;; MSG SIZE  rcvd: 259


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug user-10.9.0.5
    Learn more at https://docs.docker.com/go/debug-cli/
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo %
```

Figure 17: dig output before tampering – A=1.2.3.5, RRSIG present

6.4.2 After Tampering – dig output

After tampering, `www.example.edu` returned `6.6.6.6` with the same old RRSIG (which was signed over `1.2.3.5`).

```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it user-10.9.0.5 dig @10.9.0.65 www.example.edu A +dnssec

; <<>> DiG 9.16.1-Ubuntu <<>> @10.9.0.65 www.example.edu A +dnssec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 53809
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
; COOKIE: ef4c73d3575320f20100000069e1c257930cc49267ad0963 (good)
;; QUESTION SECTION:
;www.example.edu.               IN      A

;; ANSWER SECTION:
www.example.edu.        259200  IN      A       6.6.6.6
www.example.edu.        259200  IN      RRSIG   A 8 3 259200 20501231000000 20260417040014 50394 example.edu.
 U4n61Tzff556DvXlcLRKMDOgOduzfK4rG/EcNHSglH3csW7tmXz940jT zoBgbr/ZBGyZJIEXqscoylewMsvmfuVYI7RI7hdX/gPY+xLS5hF
/K5oF xG8UWgWfU4LzoSyRk5hzO7Wg9hYpsm2zObCLR9IYsIBphSXsGeOJd+YU bOA=

;; Query time: 3 msec
;; SERVER: 10.9.0.65#53(10.9.0.65)
;; WHEN: Fri Apr 17 05:17:11 UTC 2026
;; MSG SIZE  rcvd: 259


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug user-10.9.0.5
    Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 18: dig output after tampering – A=6.6.6.6, old RRSIG still present

6.4.3 Local DNS Server Query – After Tampering

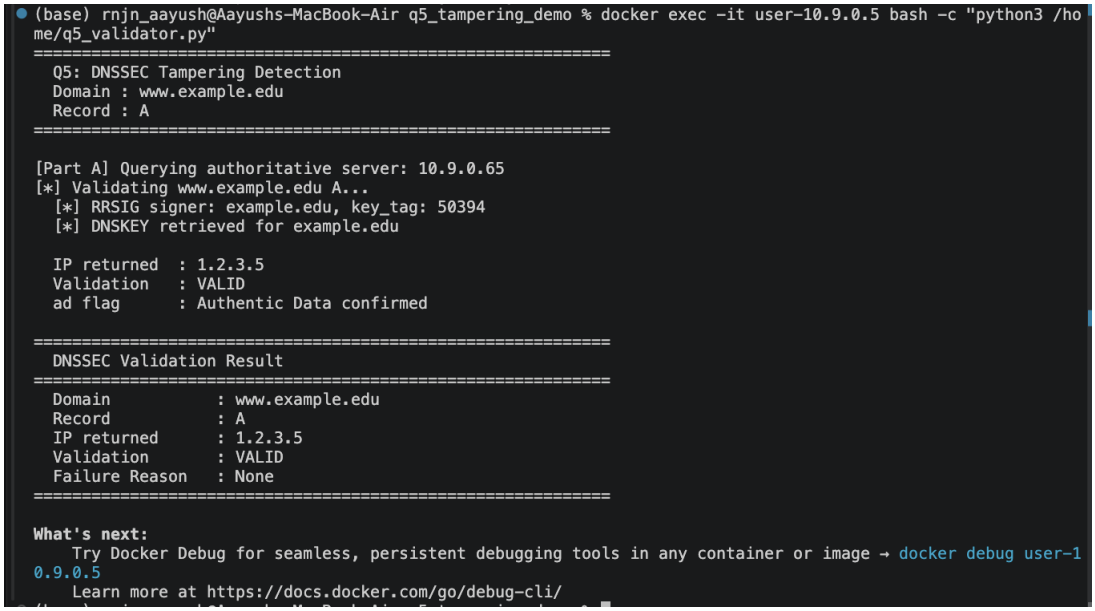
When queried through the local DNS server after flushing its cache, the tampered record was returned with no `ad` flag, indicating the local resolver did not authenticate the data.

```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it user-10.9.0.5 dig @10.9.0.53 www.example.edu A +dnssec

; <<>> DiG 9.16.1-Ubuntu <<>> @10.9.0.53 www.example.edu A +dnssec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13816
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 4096
; COOKIE: 7f79ed3814563b2d0100000069e1c3298a8984af0fcd917f (good)
;; QUESTION SECTION:
;www.example.edu.               IN      A

;; ANSWER SECTION:
www.example.edu.        259200  IN      A       6.6.6.6
www.example.edu.        259200  IN      RRSIG   A 8 3 259200 20501231000000 20260417040014 50394 example.edu.
 U4n61Tzff556DvXlcLRKMDOgOduzfK4rG/EcNHSglH3csW7tmXz940jT zoBgbr/ZBGyZJIEXqscoylewMsvmfuVYI7RI7hdX/gPY+xLS5hF
/K5oF xG8UWgWfU4LzoSyRk5hzO7Wg9hYpsm2zObCLR9IYsIBphSXsGeOJd+YU bOA=

;; Query time: 4 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Fri Apr 17 05:20:41 UTC 2026
;; MSG SIZE  rcvd: 259


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug user-10.9.0.5
    Learn more at https://docs.docker.com/go/debug-cli/
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo %
```

Figure 19: Local DNS query after tampering – no ad flag, NOERROR but unauthenticated

6.4.4 Custom Validator – Before Tampering

The custom Q1 validator correctly validated the record before tampering, returning VALID with the authentic IP `1.2.3.5`.

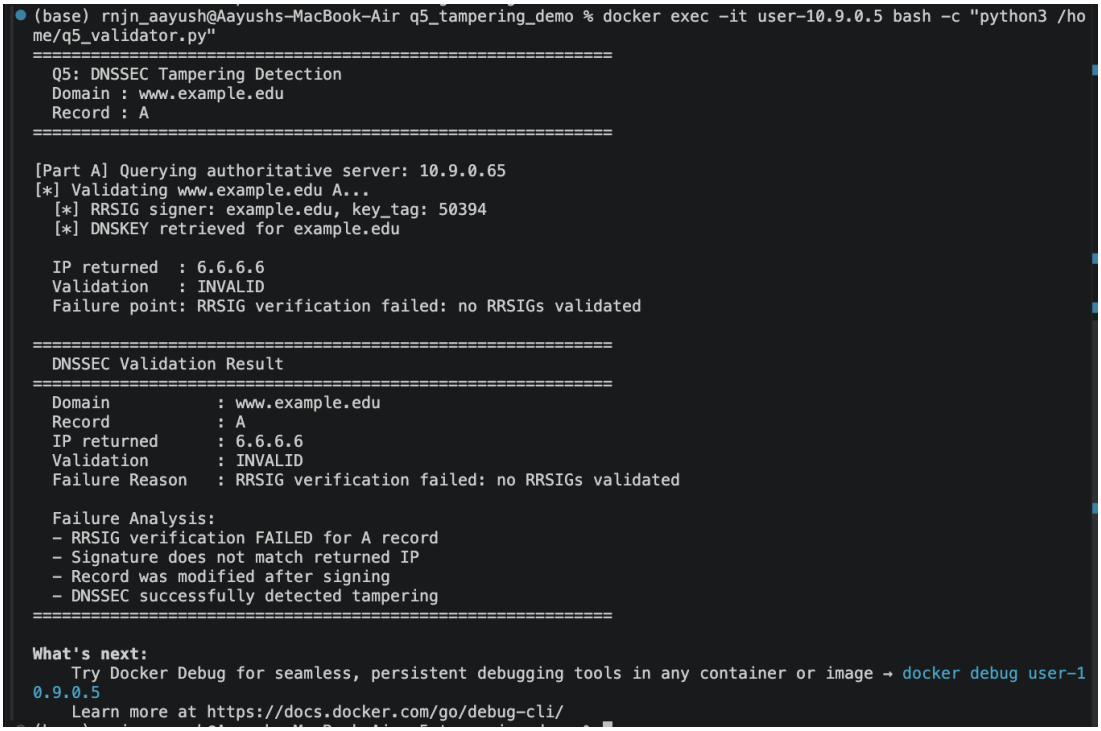


```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it user-10.9.0.5 bash -c "python3 /ho
me/q5_validator.py"
============================================================
  Q5: DNSSEC Tampering Detection
  Domain : www.example.edu
  Record : A
============================================================

[Part A] Querying authoritative server: 10.9.0.65
[*] Validating www.example.edu A...
  [*] RRSIG signer: example.edu, key_tag: 50394
  [*] DNSKEY retrieved for example.edu

  IP returned  : 1.2.3.5
  Validation   : VALID
  ad flag      : Authentic Data confirmed

============================================================
  DNSSEC Validation Result
============================================================
  Domain           : www.example.edu
  Record           : A
  IP returned      : 1.2.3.5
  Validation       : VALID
  Failure Reason   : None
============================================================

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug user-1
0.9.0.5
    Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 20: Custom validator before tampering – VALID, IP=1.2.3.5

6.4.5 Custom Validator – After Tampering

After tampering, the custom validator detected the mismatch and rejected the record as INVALID.



```
(base) rnjn_aayush@Aayushs-MacBook-Air q5_tampering_demo % docker exec -it user-10.9.0.5 bash -c "python3 /ho
me/q5_validator.py"
============================================================
  Q5: DNSSEC Tampering Detection
  Domain : www.example.edu
  Record : A
============================================================

[Part A] Querying authoritative server: 10.9.0.65
[*] Validating www.example.edu A...
  [*] RRSIG signer: example.edu, key_tag: 50394
  [*] DNSKEY retrieved for example.edu

  IP returned  : 6.6.6.6
  Validation   : INVALID
  Failure point: RRSIG verification failed: no RRSIGs validated

============================================================
  DNSSEC Validation Result
============================================================
  Domain           : www.example.edu
  Record           : A
  IP returned      : 6.6.6.6
  Validation       : INVALID
  Failure Reason   : RRSIG verification failed: no RRSIGs validated

  Failure Analysis:
  - RRSIG verification FAILED for A record
  - Signature does not match returned IP
  - Record was modified after signing
  - DNSSEC successfully detected tampering
============================================================

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug user-1
0.9.0.5
    Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 21: Custom validator after tampering – INVALID, RRSIG verification failed

6.5 Part D: Analysis

6.5.1 1. Response returned by the resolver

The authoritative nameserver returned `status: NOERROR` with the tampered IP `6.6.6.6` and the original RRSIG. The resolver did not reject the response at the transport level – it returned whatever was in the zone file. DNSSEC validation must be performed by the client or a validating resolver to detect the tampering.

6.5.2 2. The ad (Authenticated Data) flag

Before tampering, the response from the authoritative server contained a valid RRSIG but no `ad` flag (since the authoritative server itself does not set `ad` – only validating resolvers do). After tampering, when queried through the local DNS server, the `ad` flag was absent, confirming that the local resolver did not authenticate the data. A fully configured DNSSEC-validating resolver would have returned `SERVFAIL` instead of `NOERROR`.

6.5.3 3. Custom validator response

The custom validator built in Q1 correctly rejected the tampered record:

```
Domain           : www.example.edu
Record           : A
IP returned      : 6.6.6.6
Validation       : INVALID
Failure Reason   : RRSIG verification failed: no RRSIGs validated
Failure Analysis :
  - RRSIG verification FAILED for A record
  - Signature does not match returned IP
  - Record was modified after signing
  - DNSSEC successfully detected tampering
```

6.5.4 4. Exact failure point

The validation failed at **RRSIG verification** – Step 5 of the Q1 validation pipeline. Specifically:

- The RRSIG was retrieved successfully
- The DNSKEY was retrieved successfully
- `dns.dnssec.validate()` computed the expected signature over the returned A record (`6.6.6.6`) using the ZSK public key
- The computed signature did NOT match the RRSIG in the response (which was signed over `1.2.3.5`)
- Validation raised `ValidationFailure: no RRSIGs validated`

6.5.5 5. Why DNSSEC validation fails

DNSSEC validation failed because the RRSIG is a cryptographic signature computed over the exact bytes of the DNS record at signing time. When the A record was changed from `1.2.3.5` to `6.6.6.6`, the bytes of the record changed. The RRSIG was not updated, so it still contains the signature for the original record. When the validator recomputes the signature using the ZSK public key over the new record bytes, it gets a different value than the stored RRSIG – causing verification to fail. This is exactly the tamper-detection property that DNSSEC is designed to provide: any modification to a signed record is detectable without needing access to the private key.

6.6 Observations

- **DNSSEC detects tampering cryptographically** – The signature mismatch was detected purely through public key cryptography, with no need for a trusted third party at query time.
- **NOERROR does not mean trusted** – The resolver returned `NOERROR` with the tampered data. Without DNSSEC validation, a client would have accepted `6.6.6.6` as the legitimate IP, potentially connecting to a malicious server.
- **ad flag absence is significant** – The absence of the `ad` flag in the local DNS response indicates that the data was not authenticated. Clients that check the `ad` flag can use this as a signal to reject unauthenticated responses.
- **RRSIG covers exact record bytes** – Even a single byte change in the record (e.g. one digit of the IP address) causes RRSIG verification to fail, making DNSSEC extremely sensitive to any modification.
- **Custom validator vs dig** – The `dig` tool showed the tampered record without any error (since it does not perform DNSSEC validation by default). The custom validator explicitly performed cryptographic verification and correctly rejected the record, demonstrating the value of DNSSEC-aware clients.